📦 AprilTag Vision Recognition and Composite Transport (AprilTag Transport)

1. Course Content

- Master the high-level composite transport full process integrating **Nav2 autonomous navigation**, **AprilTag machine code visual pose solving**, **Ackermann chassis dual PID approach fine-tuning**, and **six-degree-of-freedom robotic arm 3D space inverse kinematics grasping**.
- Master the robotic arm base servo (0°~180°) step scanning and visual search mechanism tailored for Ackermann vehicle limited field of view.
- Master the closed-loop control of autonomous cruising, target recognition, alignment, gripper grasping transport, and return classification unloading.
- **This case uses 4 * 4 * 4 AprilTag wooden blocks**

---

2. System Architecture and Workflow

This system consists of four major modules: chassis navigation, robotic arm kinematics control, visual solving, and task state scheduling:

1. **Navigate to Operation Area**: The robot drives to the target cargo area through Nav2 autonomous obstacle avoidance; after arrival, the buzzer beeps and automatically enters visual detection ready state.
2. **Intelligent Visual Search**: Aimed at the Ackermann chassis turning radius limitation, the robotic arm base servo 1 drives the camera to perform step-by-step scanning in the 0°~180° range.
3. **Closed-Loop Approach and Fine-Tuning**: After detecting the AprilTag tag code, immediately lock the servo; through dual PID controllers, drive the chassis to fine-tune to the robotic arm's optimal grasping workspace (target distance ~190mm).
4. **Spatial Inverse Kinematics Grasping**: Through robotic arm inverse kinematics (IK) solving for six-degree-of-freedom joint target angles, execute lower gripper grasping, smoothly retract, and reset driving posture.
5. **Return to Origin and Classification Unloading**: Press the `b` key, the robot automatically plans a route back to the starting origin; after arrival, the robotic arm rotates to the corresponding shelf based on the recognized Tag ID to complete unloading.

---

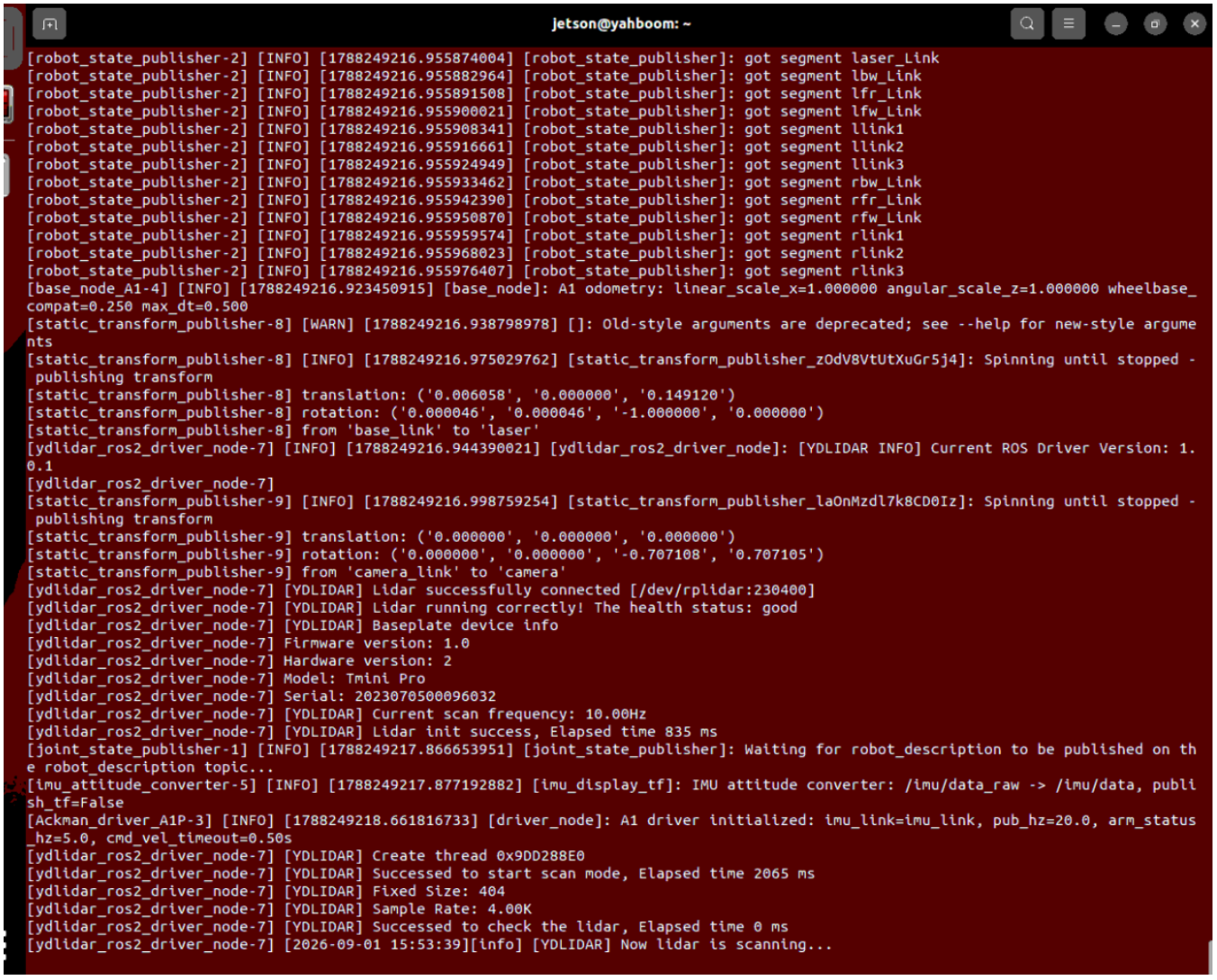
3. Preparation and Running

This case requires opening 6~8 terminals in sequence to start each functional node:

3.1 Start Chassis and Lidar

Run in **Terminal 1** (start A1 Pro chassis driver and lidar driver):

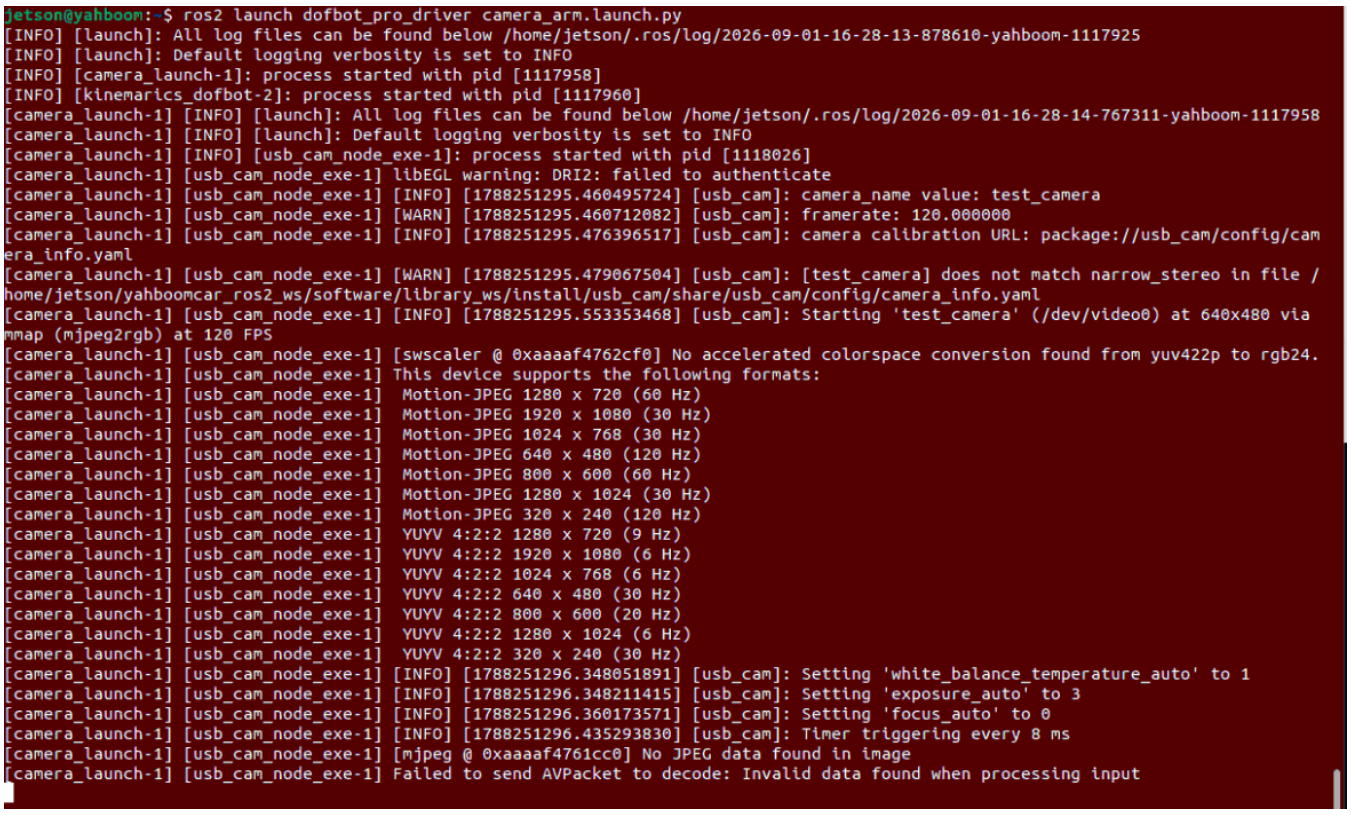
```
ros2 launch yahboomcar_nav laser_bringup_launch.py
```



3.2 Start Robotic Arm and Camera Driver

Run in **Terminal 2** (start DOFBOT Pro six-degree-of-freedom robotic arm underlying driver, forward/inverse kinematics solving service, and camera):

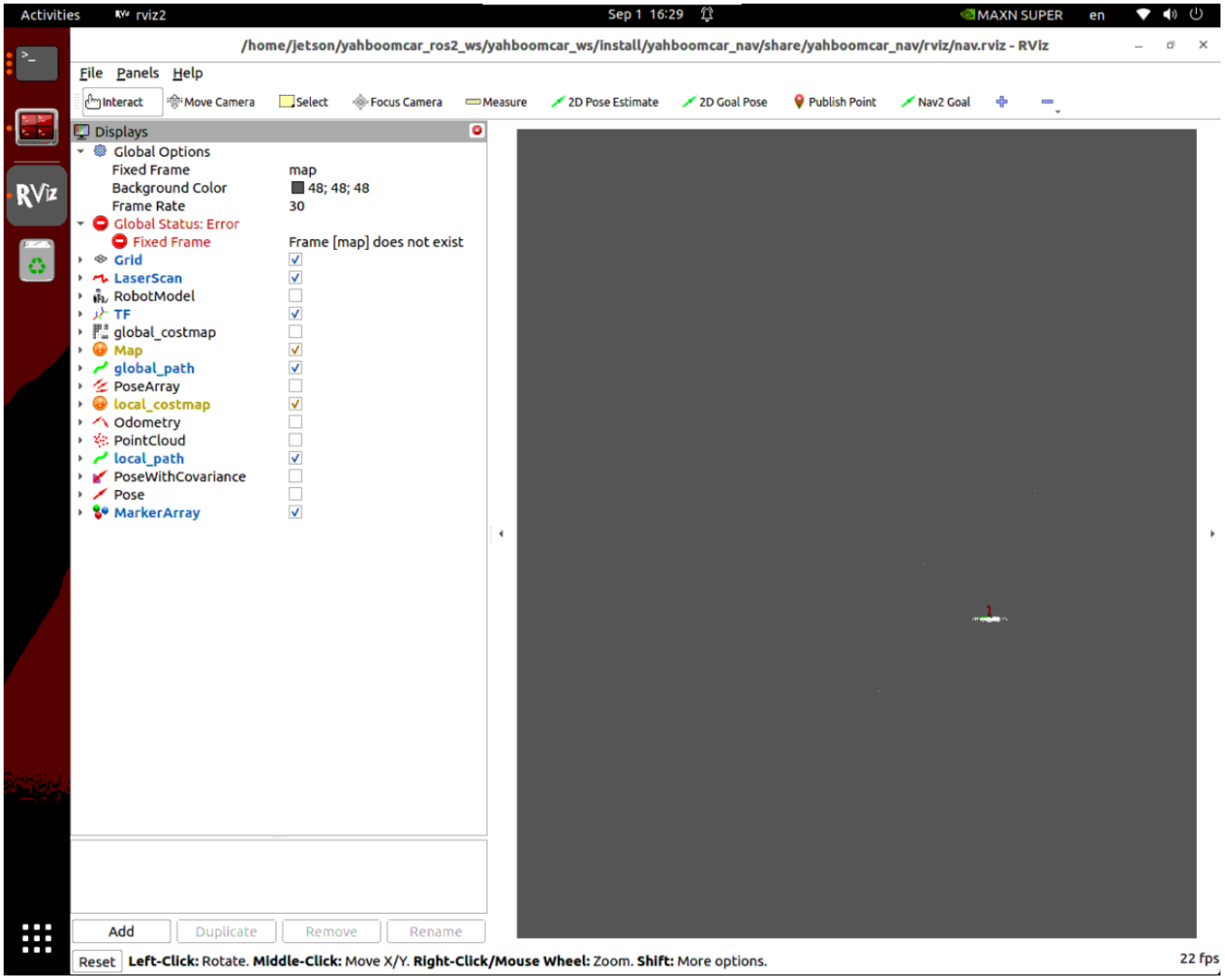
```
ros2 launch dofbot_pro_driver camera_arm.launch.py
```



3.3 Start RViz Navigation Visualization

Run in **Terminal 3** (it is recommended to open RViz on the virtual machine if the VM and robot are on the same LAN):

```
ros2 launch yahboomcar_nav display_nav_launch.py
```



3.4 Start Nav2 Autonomous Navigation Stack

Run in **Terminal 4** (start TEB local planner and global path planner):

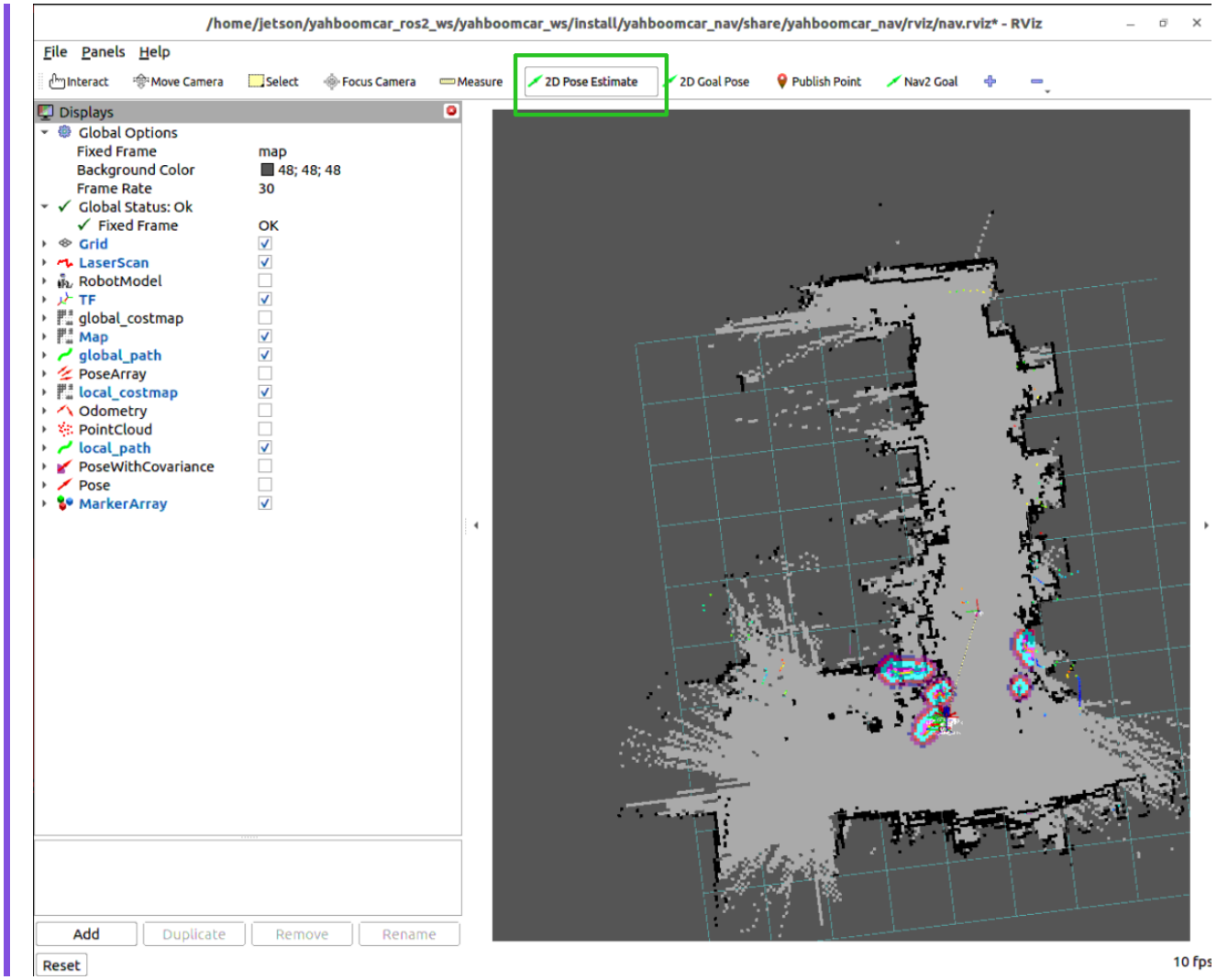
```
# Default start with map.yaml map under yahboomcar_nav
ros2 launch yahboomcar_nav navigation_teb_launch.py
```

```
jetson@yahboom: ~ 132x52
[component_container_isolated-1] [INFO] [1788251391.322590121] [bt_navigator]: Configuring
[component_container_isolated-1] [INFO] [1788251391.648996570] [lifecycle_manager_navigation]: Configuring waypoint_follower
[component_container_isolated-1] [INFO] [1788251391.649312707] [waypoint_follower]: Configuring
[component_container_isolated-1] [INFO] [1788251391.670276901] [waypoint_follower]: Created waypoint_task_executor : wait_at_waypoin
t of type nav2_waypoint_follower::WaitAtWaypoint
[component_container_isolated-1] [INFO] [1788251391.673163221] [lifecycle_manager_navigation]: Configuring velocity_smoother
[component_container_isolated-1] [INFO] [1788251391.673861416] [velocity_smoother]: Configuring velocity smoother
[component_container_isolated-1] [INFO] [1788251391.679599783] [lifecycle_manager_navigation]: Activating controller_server
[component_container_isolated-1] [INFO] [1788251391.680008594] [controller_server]: Activating
[component_container_isolated-1] [INFO] [1788251391.680095508] [local_costmap.local_costmap]: Activating
[component_container_isolated-1] [INFO] [1788251391.680140534] [local_costmap.local_costmap]: Checking transform
[component_container_isolated-1] [INFO] [1788251391.680351836] [local_costmap.local_costmap]: start
[component_container_isolated-1] [INFO] [1788251391.782204759] [controller_server]: Creating bond (controller_server) to lifecycle m
anager.
[component_container_isolated-1] [INFO] [1788251391.891012306] [lifecycle_manager_navigation]: Server controller_server connected wi
th bond.
[component_container_isolated-1] [INFO] [1788251391.891144181] [lifecycle_manager_navigation]: Activating smoother_server
[component_container_isolated-1] [INFO] [1788251391.891589602] [smoother_server]: Activating
[component_container_isolated-1] [INFO] [1788251391.891673060] [smoother_server]: Creating bond (smoother_server) to lifecycle manag
er.
[component_container_isolated-1] [INFO] [1788251391.999090233] [lifecycle_manager_navigation]: Server smoother_server connected with
 bond.
[component_container_isolated-1] [INFO] [1788251391.999240253] [lifecycle_manager_navigation]: Activating planner_server
[component_container_isolated-1] [INFO] [1788251391.999671625] [planner_server]: Activating
[component_container_isolated-1] [INFO] [1788251391.999799244] [global_costmap.global_costmap]: Activating
[component_container_isolated-1] [INFO] [1788251391.999832909] [global_costmap.global_costmap]: Checking transform
[component_container_isolated-1] [INFO] [1788251391.999867438] [global_costmap.global_costmap]: Timed out waiting for transform from
 base_footprint to map to become available, tf error: Invalid frame ID "map" passed to canTransform argument target_frame - frame do
es not exist
[component_container_isolated-1] [WARN] [1788251392.225143543] [amcl]: AMCL cannot publish a pose or update the transform. Please se
t the initial pose...
[component_container_isolated-1] [INFO] [1788251392.499940599] [global_costmap.global_costmap]: Timed out waiting for transform from
 base_footprint to map to become available, tf error: Invalid frame ID "map" passed to canTransform argument target_frame - frame do
es not exist
[component_container_isolated-1] [INFO] [1788251392.999954239] [global_costmap.global_costmap]: Timed out waiting for transform from
 base_footprint to map to become available, tf error: Invalid frame ID "map" passed to canTransform argument target_frame - frame do
es not exist
[component_container_isolated-1] [INFO] [1788251393.499941381] [global_costmap.global_costmap]: Timed out waiting for transform from
 base_footprint to map to become available, tf error: Invalid frame ID "map" passed to canTransform argument target_frame - frame do
es not exist
[component_container_isolated-1] [INFO] [1788251393.999937836] [global_costmap.global_costmap]: Timed out waiting for transform from
 base_footprint to map to become available, tf error: Invalid frame ID "map" passed to canTransform argument target_frame - frame do
es not exist
[component_container_isolated-1] [WARN] [1788251394.230184606] [amcl]: AMCL cannot publish a pose or update the transform. Please se
t the initial pose...
[component_container_isolated-1] [INFO] [1788251394.499939058] [global_costmap.global_costmap]: Timed out waiting for transform from
 base_footprint to map to become available, tf error: Invalid frame ID "map" passed to canTransform argument target_frame - frame do
es not exist
[component_container_isolated-1] [INFO] [1788251394.999963481] [global_costmap.global_costmap]: Timed out waiting for transform from
 base_footprint to map to become available, tf error: Invalid frame ID "map" passed to canTransform argument target_frame - frame do
es not exist
```

Important

Initial Pose Alignment:

1. Click the `2D Pose Estimate` tool in the top toolbar of RViz2.
2. Click at the actual robot position on the map, hold the mouse, drag the green arrow along the actual chassis heading direction and release.
3. Observe that the laser point cloud in RViz coincides with the black obstacle boundaries on the map, representing successful initial pose alignment (this position will be automatically recorded by the system as the return origin).

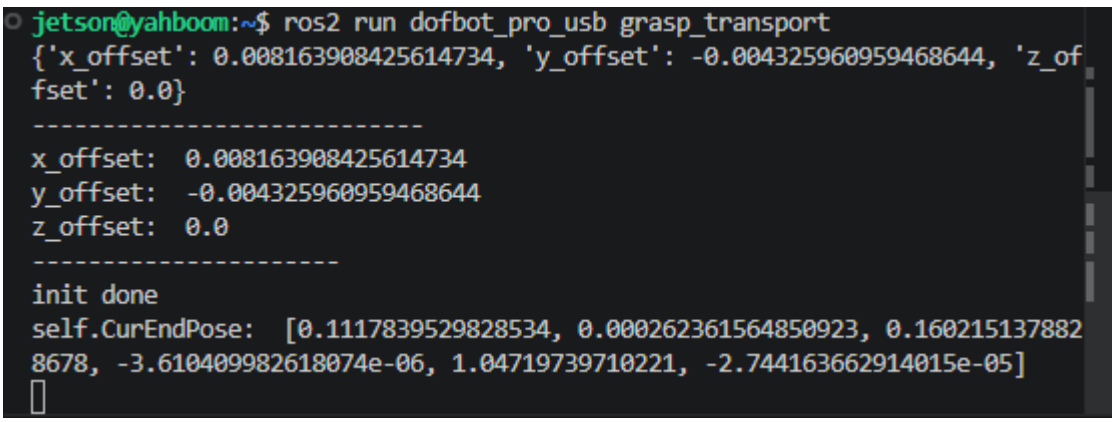


3.5 Start Robotic Arm Grasping Execution Node

Run in **Terminal 5** (load spatial inverse kinematics and grasping/unloading action sequence):

```
# Depth camera version (Nuwa)
ros2 run dofbot_pro_driver grasp_transport

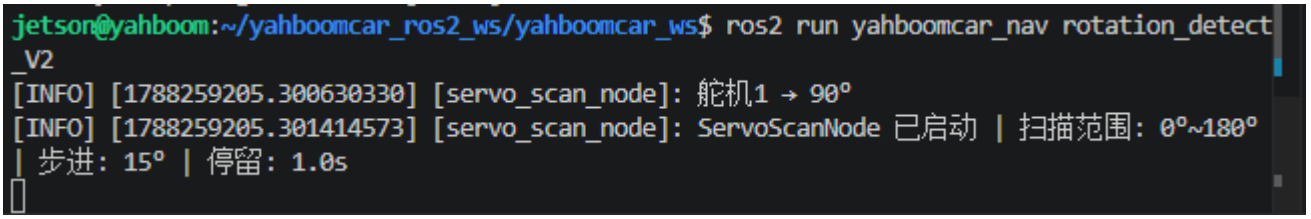
# Or USB camera version
ros2 run dofbot_pro_usb grasp_transport
```



3.6 Start Servo Scanning Search Node (Ackermann Vehicle Dedicated)

Run in **Terminal 6** (provide robotic arm base servo 0°~180° step-by-step scanning search):

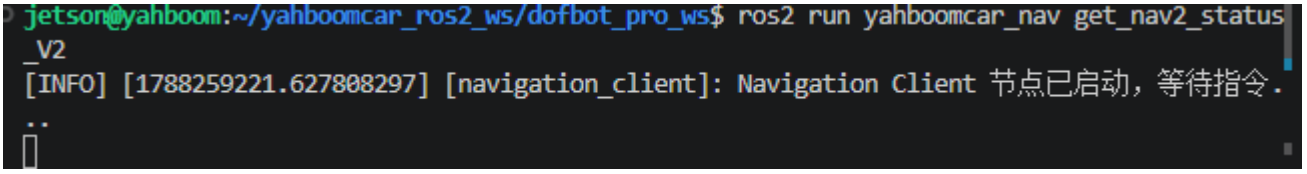
```
ros2 run yahboomcar_nav rotation_detect_V2
```



3.7 Start Navigation Status Listener and Scheduling Node

Run in **Terminal 7** (monitor arrival status, origin recording, and one-key return control):

```
ros2 run yahboomcar_nav get_nav2_status_V2
```

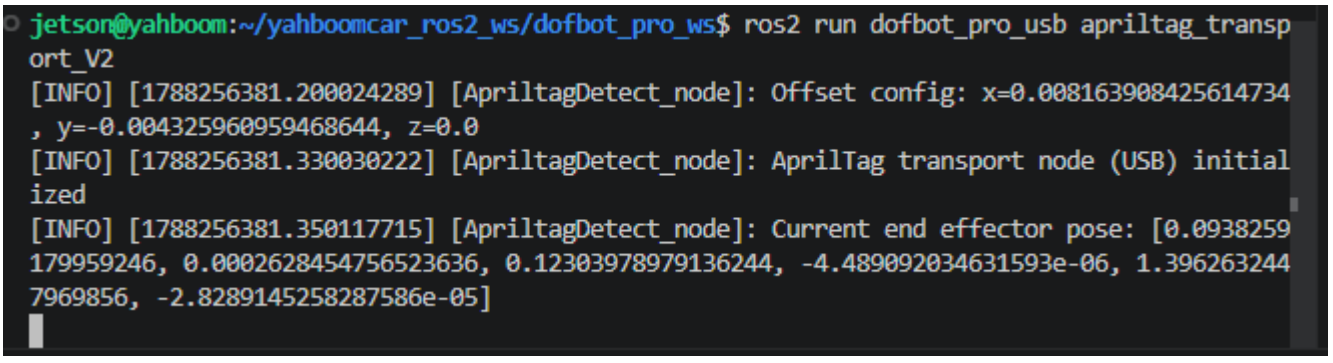


3.8 Start AprilTag Transport Master Control Decision Node

Run in **Terminal 8** (visual detection, PID approach fine-tuning, and terminal key interaction):

```
# Depth camera version (Nuwa)
ros2 run dofbot_pro_driver apriltag_transport_V2

# Or USB camera version
ros2 run dofbot_pro_usb apriltag_transport_V2
```



---

4. Operation Flow and Key Interaction Description

1. **Initial Origin Calibration**: Complete initial pose alignment in RViz using `2D Pose Estimate`, and `get_nav2_status_V2` will automatically save this coordinate as the return baseline origin.
2. **Send Target Operation Point**: Use the `2D Goal Pose` tool in RViz to specify the target area where

AprilTag machine code wooden blocks are placed on the map. The robot autonomously plans a route and obstacle-avoids to the target point; after arrival, the buzzer beeps once and the terminal prompts ready.

3. **Transport Mode Keyboard Control**: In the terminal running `apriltag_transport_V2`, press the corresponding control key:
    - **Key `r` (Search, Align, and Grasp)**:
        - If a tag code already exists in the current camera field of view, the system buzzer beeps, and the chassis immediately starts forward/backward and yaw PID fine-tuning.
        - If the tag code is not in the current field of view, automatically wake up `rotation_detect_V2` to drive robotic arm base servo 1 to step-scan between 0°~180°; once the target is detected, immediately lock the servo.
        - The chassis automatically adjusts posture and distance, approaching to the optimal grasping distance (~190mm).
        - The robotic arm automatically lowers the gripper according to spatial inverse kinematics, closes the gripper to firmly hold the wooden block, retracts and lifts to safe driving height.
    - **Key `b` (One-Key Return and Classification Unloading)**:
        - The robot automatically calls Nav2 Action to return to the initial origin.
        - After arriving at the origin, the robotic arm automatically rotates to the corresponding cargo slot angle based on the wooden block's Tag ID (1~4) and lowers the gripper to release and complete unloading.
    - **Key `n` (Continue Cruising)**:
        - If the target is not found at the current point or continuous cruising needs to be executed, send the next goal point in RViz.

> **Note**
>
> When you need to end the program, press `Ctrl+C` in each terminal window to stop the program.

---

5. Core Source Code and Key Implementation Mechanism Analysis

5.1 Transport Master Control and Chassis PID Approach (apriltag_transport_V2.py)

Source Code Path:

```
$HOME/yahboomcar_ros2_ws/dofbot_pro_ws/src/dofbot_pro_usb/dofbot_pro_usb/apriltag_transport_V2.py
```

1. Spatial Pose Estimation and Chassis Dual PID Fine-Tuning

The node uses camera intrinsic parameters and AprilTag corners to calculate the target object's distance and yaw error in space, and through linear and angular velocity dual PID control, smoothly approaches the chassis:

```
def _process_tag_tracking(self, tags, result_image, fx, fy, cx0, cy0):
    for tag_current in tags:
        center_x, center_y = tag_current.center

        # 1. Solve 3D coordinates of tag code in camera coordinate system
        cam_coords = self.estimate_tag_camera_coords(tag_current, fx, fy, cx0, cy0)
        pose = self.compute_world_pose(cam_coords)
        dist_detect = math.sqrt(pose[1] ** 2 + pose[0] ** 2) * 1000.0  # millimeters

        # 2. Arm center reset (when approaching 250~265mm)
        if not self.arm_centered and 250.0 < dist_detect < 265.0:
            self.pubVel(0, 0, 0)
            self.pubSix_Arm(self.init_joints)
            self.arm_centered = True

        # 3. Y-axis lateral alignment (correct angular velocity through PID)
        if self.y_track:
            err_pixel_x = center_x - TARGET_PIXEL_X
            if abs(err_pixel_x) > 10:
                self.angle_pid.SetPoint = TARGET_PIXEL_X
                self.angle_pid.update(center_x)
                angular_z = float(self.angle_pid.output)
                self.pubVel(0, 0, angular_z)
            else:
                self.pubVel(0, 0, 0)
                self.y_track = False
                self.adjust_dist = True

        # 4. X-axis distance approach (target distance TARGET_DIST_MM = 190mm)
        if self.adjust_dist:
            dist_err = dist_detect - TARGET_DIST_MM
            if abs(dist_err) > DIST_THRESHOLD_MM:
                self.linearx_pid.SetPoint = TARGET_DIST_MM
                self.linearx_pid.update(dist_detect)
                linear_x = -float(self.linearx_pid.output)
                self.pubVel(linear_x, 0, 0)
            else:
                # Approach complete, stop chassis and trigger robotic arm inverse kinematics
grasping
                self.pubVel(0, 0, 0)
                self.adjust_dist = False
                self.grasp(tag_current.tag_id, pose)
```

2. Non-Blocking Terminal Keyboard Listening Mechanism

Uses cross-platform independent thread (Linux environment uses `termios` and `select` non-canonical mode) to capture terminal keyboard key inputs (`r`/`n`/`b`), and puts key events into a queue processed by ROS 2 timer polling, ensuring immediate response of human-machine interaction and efficient decoupling from the main control loop.

5.2 Servo Step-by-Step Scanning Search Mechanism (rotation_detect_V2.py)

Source Code Path:

```
$HOME/yahboomcar_ros2_ws/yahboomcar_ws/src/yahboomcar_nav/yahboomcar_nav/rotation_detect_V2.py
```

Due to Ackermann chassis steering structure limitations, it cannot perform in-place 360° rotation like differential/omni-wheel chassis. This system designs `ServoScanNode`, which controls the robotic arm base servo 1 to drive the camera for multi-step scanning:

```python
class ServoScanNode(Node):
    def start_rotate_callback(self, msg):
        """Receive scan command, servo gradually rotates from scan_start (0°) to scan_end (180°)"""
        self.scanning = True
        self.detected = False
        self.current_angle = self.scan_start
        self.move_servo(self.scan_start)

        # Timed step scanning (each step step_pause = 1.0s)
        wait_sec = self.servo_speed / 1000.0 + self.step_pause
        self.scan_timer = self.create_timer(wait_sec, self.scan_step_callback)

    def detect_result_callback(self, msg):
        """Visual node recognized target, immediately lock scanning ahead of schedule"""
        if msg.data and self.scanning:
            self.detected = True
            self.stop_scan(found=True)
```

5.3 Robotic Arm Spatial Inverse Kinematics Grasping and Classification Unloading (grasp_transport.py)

Source Code Path:

```
$HOME/yahboomcar_ros2_ws/dofbot_pro_ws/src/dofbot_pro_usb/dofbot_pro_usb/grasp_transport.py
```

- **3D Inverse Kinematics Solving**: Receives material spatial coordinates `(x, y, z)` from the visual node, combined with the robotic arm and camera extrinsic calibration transformation matrix `EndToCamMat`

and bias configuration file, calls the underlying `get_kinemarics` inverse solving service to calculate the 6 servo target pulse width angles.

- **Step-by-Step Safe Grasping Trajectory**:
    1. Gripper fully opens, robotic arm moves to the material upper transition point.
    2. Smoothly vertically descends to grasping height, closes gripper to clamp the wooden block.
    3. Vertically lifts and retracts to safe driving posture, publishes `/grasp_done` signal.
- **Multi-Tag Classification Unloading**: Based on different `tag_id`, preset corresponding unloading joint angle dictionary `TAG_UNLOAD_POSITIONS`; after returning to origin, accurately delivers the wooden block to the specified cargo slot.

---

6. Core Parameter Description

Parameter Module	Parameter Name	Current Config Value	Core Function and Physical Meaning
Visual Detection	`families`	`tag36h11`	AprilTag dictionary family (extremely strong anti-false-detection ability)
Visual Detection	`tag_size`	`0.025`	AprilTag wooden block tag physical side length (meters)
Chassis Approach	`TARGET_DIST_MM`	`190.0`	Robotic arm optimal grasping longitudinal target distance (millimeters)
Chassis Approach	`DIST_THRESHOLD_MM`	`5.0`	Allowable dead zone tolerance for grasping distance arrival (millimeters)
Chassis Approach	`TARGET_PIXEL_X`	`341`	Image center X pixel coordinate for target alignment
Chassis PID	`LINEAR_X_PID`	`(0.5, 0.0, 0.2)`	Forward/backward distance approach PID parameters (output linear velocity v_x)
Chassis PID	`ANGLE_PID`	`(0.5, 0.0, 0.2)`	Lateral angle alignment PID parameters (output angular velocity ω_z)
Servo Scanning	`scan_start` / `scan_end`	`0° / 180°`	Robotic arm servo 1 search scanning angle range
Servo Scanning	`scan_step` / `step_pause`	`15° / 1.0s`	Each scanning step length and dwell time for visual detection
Offset Calibration	`x/y/z_offset`	Dynamically loaded	Spatial micro-adjustment compensation of gripper and material grasping geometric center (YAML configuration)